Advanced Programming
2nd Project: Functional Programming
Deadline: 17 April 2026, 23:59

Programming Group
guido.salvaneschi@unisg.ch

University of Zurich

1 Assignment

In this assignment, we will implement in Haskell our own small SAT solver: a program which takes a logical formula such as $(x \wedge y) \wedge (\neg x \vee \neg y)$ and returns an assignment for x and y that makes the formula true (if it exists). Below, the first two sections are independent from each other, in the third we will glue all the definitions together into a working minimal SAT solver. After you have completed the assignment, please submit your solution using the course website. You should submit ONLY one file, called `Assignment.hs`.

1.1 Finding All Possible Assignments

Our SAT solver will not be smart. It will try all the possible assignment of all the variables, one-by-one, until a solution is found.

The main part of the function that generates "all the possible assignment" is the Cartesian product. In its basic form, the Cartesian product $A \times B$ of two sets (or lists) A and B is the set (or list) that contains all the possible pairs whose left-hand-side comes from A and its right-hand-side comes from B. For example:

$$\{1, 2\} \times \{a, b, c\} = \{(1, a), (1, b), (1, c), (2, a), (2, b), (2, c)\}$$

Task 1: Cartesian Product

Implement the function `cartesian :: [a] -> [b] -> [(a, b)]` that takes two lists and returns the one that contains all the pairs from their Cartesian product.

Now, `cartesian [True, False] [True, False]` can find all the possible values of two boolean variables. Next, we want to generalize the Cartesian product for an arbitrary number of inputs. To appease the type-checker we will make the compromise that the sets (or lists) will all contain elements of the same type. Before we implement this generalized Cartesian product, we will define the following useful helper function.

Task 2: List of Pairs to List of Lists

Implement the function `lpl2ll :: [(a, [a])] -> [[a]]` which, for every pair in the list, sets the left-hand-side to the head of the right-hand-side.

For example `lpl2ll [(1,[2]), (3,[4,5])] == [[1,2], [3,4,5]]`.

Now we are able to implement the generalized Cartesian product.

Task 3: Generalized Cartesian Product

Implement the function `genCartesian :: [[a]] -> [[a]]` using `cartesian` and `lpl2ll`.

For example:

- `genCartesian [] == [[]]`
- `genCartesian [[1],[2,3],[4,5,6]]`
  `== [[1,2,4],[1,2,5],[1,2,6],[1,3,4],[1,3,5],[1,3,6]]`

Finally, for this section, we want a function that takes a list of variable names and returns all the possible assignments of these variables to `True` or `False`. We will represent this assignment as simply a pairing. Therefore, if we see the list `[("x",True),("y",False)]` we know that it's representing the assignments `x:=True; y:=False`.

Task 4: Find All Possible Assignments

Implement the `allAssignments :: [String] -> [[(String,Bool)]]` function, which takes a list of variable names and returns all the possible assignments.

For example:

```
allAssignments ["x", "y"] == [
  [("x",True),  ("y",True)],
  [("x",True),  ("y",False)],
  [("x",False), ("y",True)],
  [("x",False), ("y",False)]
]
```

1.2 Encoding a Formula

The logical formulas we will be interesting in checking abide by the following strict grammar:

```
<formula> ::= <formula> "and" <formula>
            | <formula> "or" <formula>
            | <formula> "implies" <formula>
            | "not" <formula>
            | <variable>
```

The `<variable>` rule stands for any string, for example `x` or `isRaining`.

Task 5: Formula Data Type

In the course we have seen how to Algebraic Data Types (ADTs) can represent a grammar. Define the `data Formula` ADT with the necessary constructors that match the grammar of `<formula>`. For example, to encode the formula $(p \wedge (q \vee p)) \rightarrow \neg s$ the following code MUST compile as-is:

```
formula = Implies (And (Var "p") (Or (Var "q") (Var "p"))) (Not (Var "s"))
```

When we search for a solution, we want a way to collect all a list of all the unique variables that occur in the formula.

Task 6: Formula Variables

Implement the `vars :: Formula -> [String]` that collects all the variables that occur in the formula.

Hint: it is okay if a variable occurs more than once in the list.

Task 7: Unique Elements in a List

Implement the `unique :: Eq a => [a] -> [a]` function that removes duplicates from a list.

Hint: the definition will be easier if we define the `contains :: Eq a => a -> [a] -> Bool` helper function that checks membership in a list.

1.3 Finding a Satisfying Assignment

Now we are ready to fit all our definitions together to first check if a given assignment satisfies the formula (i.e. it makes it true), and then searching through all the valid assignments if any satisfies the formula.

Task 8: Check Assignment

Implement the `check :: Formula -> [(String,Bool)] -> Bool` function that takes an assignment and a formula and returns `True` if the assignment satisfies the formula and `False` otherwise. If a variable in the formula does not occur in the list of assignments we may assume it is assigned to `False`.

Hint 1: the implication $p \to q$ is equivalent to $\neg p \vee q$.
Hint 2: define the `valueOf :: String -> [(String,Bool)] -> Bool` helper function which returns the boolean value associated with a variable name in a list of assignments.

Finally, using all the previous functions which we have defined, we are ready to implement the solver.

Task 9: Solving a Formula

Implement the `solve :: Formula -> Maybe [(String,Bool)]` which takes a formula and returns, if it exists, an assignment that makes it satisfiable.

Now let's put our solver to work by checking if some formulas are satisfiable or not.

Task 10: Use the Solver

Which of the following formulas is satisfiable? And what is an assignment that satisfies it? Encode the following formulas and check them with your solver.

1. $a \vee b \vee c \vee d \vee e \vee f$
2. $\neg a \wedge a \wedge b \wedge c \wedge d \wedge e \wedge f$